
title: 'Get Started with DVC' description:
'Get a quick introduction to the major features of DVC for data science and machine learning projects: version data, access it anywhere, capture pipelines and metrics, and manage experiments.'

Get Started with DVC

► Details

How cool would it be to track large datasets and machine learning models alongside your code, sidestepping all the limitations of storing it in Git? Imagine cloning a repository and immediately seeing your datasets, checkpoints and models staged in your workspace. Imagine switching to a different version of a 100Gb file in less than a second with a `git checkout`.

□ *DVC is your "Git for data"!*

Installing DVC

First things first, let's install DVC. The easiest way to install dvc is with [uv](https://docs.astral.sh/uv/) (<https://docs.astral.sh/uv/>) or [pipx](https://pipx.pypa.io/stable/) (<https://pipx.pypa.io/stable/>):

```
$ uv tool install dvc # or,  
$ pipx install dvc
```

Once it's installed, verify dvc is available by running:

```
$ dvc --help
```

If you see a help message listing dvc commands, you are good to continue!

See the [Installation Guide \(/install\)](#) for more installation options.

Initializing a project

Before we begin, settle on a directory for this guide. Everything we will do will be self contained there.

► Details

Inside your chosen directory, we will use our current working directory as a DVC project. Let's initialize it by running `dvc init` inside a Git project:

```
$ dvc init
```

A few [internal files \(/user-guide/project-structure/internal-files\)](#) are created that should be added to Git:

```
$ git status
Changes to be committed:
  new file:   .dvc/.gitignore
  new file:   .dvc/config
  ...
$ git commit -m "Initialize DVC"
```

Now you're ready to DVC!

Tracking data

Working inside an [initialized project \(/start#initializing-a-project\)](#) directory, let's pick a piece of data to work with. We'll use an example `data.xml` file, though any text or binary file (or directory) will do. Start by running:

```
$ dvc get https://github.com/treeverse/dataset-registry \
  get-started/data.xml -o data/data.xml
```

We used `dvc get` above to show how DVC can turn any Git repo into a "[data registry \(/example-scenarios/data-registry\)](#)". `dvc get` can download any file or directory tracked in a DVC repository.

Use `dvc add` to start tracking the dataset file:

```
$ dvc add data/data.xml
```

DVC stores information about the added file in a special `.dvc` file named `data/data.xml.dvc`. This small, human-readable metadata file acts as a placeholder for the original data for the purpose of Git tracking.

Next, run the following commands to track changes in Git:

```
$ git add data/data.xml.dvc data/.gitignore
$ git commit -m "Add raw data"
```

Now the *metadata about your data* is versioned alongside your source code, while the original data file was added to `.gitignore`.

► Details

Storing and sharing

You can upload DVC-tracked data to a variety of storage systems (remote or local) referred to as "[remotes](/user-guide/data-management/remote-storage)". For simplicity, we will use a "local remote" for this guide, which is just a directory in the local file system.

Configuring a remote

Before pushing data to a remote we need to set it up using the `dvc remote add` command:

```
$ mkdir /tmp/dvcstore
$ dvc remote add -d myremote /tmp/dvcstore
```

```
$ mkdir %TEMP%/dvcstore
$ dvc remote add -d myremote %TEMP%\dvcstore
```

DVC supports many remote [storage types](/user-guide/data-management/remote-storage#supported-storage-types), including Amazon S3, NFS, SSH, Google Drive, Azure Blob Storage, and HDFS.

An example for a common use case is configuring an [Amazon S3](/user-guide/data-management/remote-storage/amazon-s3) remote:

```
$ dvc remote add -d storage s3://mybucket/dvcstore
```

For this to work, you'll need an AWS account and credentials set up to allow access.

To learn more about storage remotes, see the [Remote Storage Guide](/user-guide/data-management/remote-storage).

Uploading data

Now that a storage remote was configured, run `dvc push` to upload data:

```
$ dvc push
```

► Details

Usually, we would also want to Git track any code changes that led to the data change (`git add`, `git commit` and `git push`).

Retrieving data

Once DVC-tracked data and models are stored remotely, they can be downloaded with `dvc pull` when needed (e.g. in other copies of this project). Usually, we run it after `git pull` or `git clone`.

Let's try this now:

```
$ dvc pull
```

► Details

Making local changes

Next, let's say we obtained more data from some external source. We will simulate this by doubling the dataset contents:

```
$ cp data/data.xml /tmp/data.xml
$ cat /tmp/data.xml >> data/data.xml
```

```
$ copy data\data.xml %TEMP%\data.xml
$ type %TEMP%\data.xml >> data\data.xml
```

After modifying the data, run `dvc add` again to track the latest version:

```
$ dvc add data/data.xml
```

Now we can run `dvc push` to upload the changes to the remote storage, followed by a `git commit` to track them:

```
$ dvc push
$ git commit data/data.xml.dvc -m "Dataset updates"
```

Switching between versions

A commonly used workflow is to use `git checkout` to switch to a branch or checkout a specific `.dvc` file revision, followed by a `dvc checkout` to sync data into your workspace:

```
$ git checkout <...>
$ dvc checkout
```

Return to a previous version of the dataset

Let's go back to the original version of the data:

```
$ git checkout HEAD~1 data/data.xml.dvc
$ dvc checkout
```

Let's commit it (no need to do `dvc push` this time since this original version of the dataset was already saved):

```
$ git commit data/data.xml.dvc -m "Revert dataset updates"
```

As you can see, DVC is technically not a version control system by itself! It manipulates `.dvc` files, whose contents define the data file versions. Git is already used to version your code, and now it can also version your data alongside it.

Following This Guide

To help you understand and use DVC better, consider the following three example scenarios: **data pipelines**, **experiment tracking** and **model management**. You may pick any to start learning about how DVC helps you "solve" that scenario!

Choose a trail to jump into its first chapter:

- **Data Pipelines** (</start/data-pipelines/data-pipelines>) - Use DVC as a build system for reproducible, data driven pipelines.
- **Experiment Management** (</start/experiments/experiment-tracking>) - Easily track your experiments and their progress by only instrumenting your code, and collaborate on ML experiments like software engineers do for code.
- **Model Registry** - Use the DVC model registry to manage the lifecycle of your models in an auditable way. Easily access your models and integrate your model registry actions into CI/CD pipelines to follow GitOps best practices.